

A
Project Report
on
NGINX Anomaly Detection using Isolation Forest

Submitted By
BESTHAVEMULA VARSHITHA [R210036]
RAJA KAMALANI [R210603]
MASA NANI [R210612]

Under the Esteemed Guidance of
Dr. PENUGONDA RAVI KUMAR
M.E. (IISc Bangalore), Ph.D. (University of Aizu, Japan)
Assistant Professor
Department of Computer Science and Engineering



RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES
(APIIT)
R K VALLEY, VEMPALLI, YSR (Dist.) – 516330
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
2025-2026



**RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES(AP
IIIT)**

R.K Valley, Vempalli(M), Kadapa(Dist) – 516330

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING 2025-2026

CERTIFICATE OF PROJECT COMPLETION

This is to certify that the project report entitled "NGINX Anomaly Detection using Isolation Forest" being submitted by BESTHAVEMULA VARSHITHA(R210036), RAJA KAMALANI(R210603) , MASA NANI(R210612) under the guidance and supervision of

Dr. Penugonda Ravi Kumar is submitted to the DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING in partial fulfilment of the requirements for the award of Bachelor of Technology in Computer Science and Engineering during the academic year 2025-2026 and it has been found worthy of acceptance according to the requirements of the University.

Project Guide

Dr. Penugonda Ravi Kumar

M.E. (IISc Bangalore),

Ph.D. (University of Aizu, Japan)

Assistant Professor

Department of CSE

RGUKT-RK Valley

Head Of The Department (CSE)

Dr. Ch. Ratna Kumari

M.Tech. (University of Hyderabad)

Ph.D. (JNTU Hyderabad)

Head of the Department

Computer Science and Engineering

RGUKT RK VALLEY

Signature of External Examiner



**RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES(AP
IIIT)**

R.K Valley, Vempalli(M), Kadapa(Dist) – 516330

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING 2025-2026

DECLARATION

We, Besthavemula Varshitha [R210036], Raja Kamalani [R210603] and Masa Nani [R210612] hereby declare that the project report entitled "NGINX Anomaly Detection using Isolation Forest" done by us under the guidance of Dr. Penugonda Ravi Kumar is submitted in partial fulfillment for the degree of Bachelor of Technology in Computer Science and Engineering during the academic session December 2025 – April 2026 at RGUKT-RK Valley.

We also declare that this project is a result of our own effort and has not been copied or imitated from any source. Citations from research papers and websites are mentioned in the references. To the best of our knowledge, the results embodied in this project report have not been submitted to any university or institute for the award of any degree or diploma.

Date:

Place: RGUKT-RK Valley

BESTHAVEMULA VARSHITHA [R210036]

RAJA KAMALANI [R210603]

MASA NANI [R210612]



**RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES(AP
IIIT)**

R.K Valley, Vempalli(M), Kadapa(Dist) – 516330

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING 2025-2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and deep respect to all those who guided, inspired and helped us to successfully complete this Mini Project titled "NGINX Anomaly Detection using Isolation Forest."

We are deeply thankful to our project guide, Dr. Penugonda Ravi Kumar, M.E. (IISc Bangalore), Ph.D. (University of Aizu, Japan), for his invaluable guidance, constant encouragement, constructive suggestions and mentorship at every stage of the project. His expertise and support played a crucial role in shaping the direction and outcome of our work.

We extend our heartfelt thanks to Dr. Ch. Ratna Kumari, M.Tech. (University of Hyderabad), Ph.D. (JNTU Hyderabad), Head of the Department, Computer Science and Engineering, for her continuous support, encouragement, and for providing us with the resources and academic environment necessary for the completion of this project.

We sincerely acknowledge and express our thankfulness to Mr. D. Ramesh Kailas, M.Tech., Dean of Academics, RGUKT RK Valley, for his academic support and encouragement throughout the project.

We are also grateful to Prof. A. V. S. S. Kumara Swami Gupta, M.Tech., Ph.D., Director of RGUKT RK Valley, for his encouragement and for fostering a culture of research and innovation within the university.

Our sincere thanks also go to the faculty members of the Department of Computer Science and Engineering for their academic guidance and continued support.

With Sincere Regards,

BESTHAVEMULA VARSHITHA [R210036]

RAJA KAMALANI [R210603]

MASA NANI [R210612]



**RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES(AP
IIIT)**

R.K Valley, Vempalli(M), Kadapa(Dist) – 516330

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING 2025-2026

TABLE OF CONTENTS

Chapter Name	Page Number
Abstract	V
Chapter 1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope of the Project	2
1.5 Existing Solutions	3
1.6 Proposed Solution	3
Chapter 2 Literature Review	4
2.1 Web Server Log Analysis	4
2.2 Empirical Log Anomaly Detection	5
2.3 NGINX-ADS Framework	5
2.4 ML-Based Log File Analysis	6
2.5 Isolation Forest Algorithm	6
2.6 Gaps Identified	7
Chapter 3 System Design	8
3.1 System Overview	8
3.2 Architecture	8
3.3 Data Flow	9
3.4 Tools and Technologies	10

3.5 Design Considerations	11
Chapter 4 Implementation	12
4.1 Log Parsing and Feature Extraction	12
4.2 Feature Engineering	12
4.3 Model Training	13
4.4 FastAPI Backend	13
4.5 Real-Time Log Watcher	14
4.6 Alert System	15
Chapter 5 Results and Discussion	16
5.1 Experimental Setup	16
5.2 Anomaly Detection Results	16
5.3 System Screenshots	17
5.4 Performance Comparison	19
5.5 Qualitative Analysis	20
Chapter 6 Conclusion and Future Work	21
6.1 Conclusion	21
6.2 Future Work	21
References	23



**RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES(AP
IIIT)**

R.K Valley, Vempalli(M), Kadapa(Dist) – 516330

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING 2025-2026

ABSTRACT

Web server log analysis has emerged as a critical pillar of modern network security engineering, with NGINX serving as one of the most widely deployed reverse proxy and web server platforms across enterprise infrastructures. As cyber threats grow in sophistication and volume, traditional threshold-based and rule-driven anomaly detection mechanisms have proven inadequate in identifying novel, subtle, and distributed attack vectors concealed within the high-throughput log streams that NGINX generates.

This project presents *noginx* — a real-time NGINX anomaly detection and alerting system that leverages the Isolation Forest machine learning algorithm to identify anomalous HTTP request patterns in live NGINX web server logs. The system operates as a lightweight two-component architecture: a FastAPI-based inference backend serving a pre-trained Isolation Forest model, and a Python-based log watcher that monitors `/var/log/nginx/access.log` in real time, extracts structured features from each log entry, queries the backend for anomaly classification, and triggers desktop notifications along with email alerts upon detection of anomalous requests.

The Isolation Forest model is trained on a publicly available NGINX logs dataset with six features: HTTP method, request path, response status code, response size, user agent identifier, and hour of day. Real-time testing confirmed successful detection of anomalous access patterns with immediate alert delivery via desktop notification and email. The system achieves an estimated F1-score of approximately 0.76, demonstrating competitive performance while operating in a fully unsupervised manner.

Keywords: NGINX Log Analysis, Anomaly Detection, Isolation Forest, FastAPI, Real-Time Monitoring, Web Server Security, Python, Machine Learning

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The modern internet is underpinned by an intricate web of server infrastructure, load balancers, and reverse proxies that mediate billions of client-server interactions every day. Among these, NGINX has established itself as one of the most widely adopted high-performance web servers globally, powering everything from small enterprise applications to hyperscale deployments serving tens of millions of concurrent users. Its asynchronous, event-driven architecture makes it exceptionally efficient at processing enormous volumes of HTTP traffic and, as a direct consequence, produces enormously detailed access and error log streams that encapsulate a near-complete record of every client interaction.

Every entry in an NGINX access log carries structured data: the requesting IP address, the HTTP method employed, the requested URI, the server response status code, the bytes transferred, and the user-agent string. Within this vast stream of nominal traffic lie the fingerprints of malicious actors — automated bots, credential stuffing attacks, distributed denial-of-service floods, and reconnaissance activities probing server configurations for exploitable vulnerabilities.

The OWASP Foundation has documented that the majority of web application security incidents are preceded by detectable behavioral signals in server logs that go unnoticed due to insufficient analytical tooling. Research indicates that approximately 84% of web applications carry at least one medium-severity vulnerability, underscoring the critical importance of continuous, intelligent log monitoring.

1.2 Problem Statement

The fundamental challenge of NGINX anomaly detection is that signals of malicious activity are rarely overt. A DDoS attack in its early stages may be statistically indistinguishable from a legitimate traffic surge. Rule-based and threshold-based detection systems — the most prevalent approach in production environments today — are fundamentally insufficient because they require prior knowledge of attack signatures and fail to generalize to novel attack patterns.

Furthermore, the volume of NGINX log generation in production environments renders manual log inspection entirely impractical. Security engineers require automated, real-time detection with immediate alerting mechanisms for rapid incident response. The absence of labeled anomaly data in most production environments also eliminates supervised learning approaches as a practical solution, demanding unsupervised techniques that can detect novelty without ground-truth annotations.

1.3 Objectives

The primary objectives of this project are:

- To implement a real-time NGINX log monitoring system capable of processing live log entries as they are written by the NGINX server process.
- To apply the Isolation Forest unsupervised machine learning algorithm for anomaly detection in NGINX access log feature spaces without requiring labeled training data.
- To develop a lightweight modular two-component architecture: a FastAPI-based ML inference backend and a Python-based real-time log watcher.
- To implement an automated alert delivery system that notifies security personnel via desktop notifications and email with full anomaly context upon detection.
- To support custom model retraining on organization-specific NGINX log data for improved detection accuracy in production deployments.

1.4 Scope of the Project

This project focuses on the implementation of a practical, deployable anomaly detection system for NGINX web server access logs on Unix-based operating systems. The system monitors the standard `/var/log/nginx/access.log` file in real time alongside a running NGINX process. The scope includes log parsing and feature extraction, ML model training using Isolation Forest, REST API development using FastAPI, real-time log tailing and anomaly classification, and multi-channel alert delivery.

The project does not include a graphical dashboard or enterprise SIEM integration in its current iteration; however, these are identified as high-priority future work directions. The scope is deliberately constrained to ensure a fully functional, tested, and deployable core system within the Mini Project duration.

1.5 Existing Solutions

Several tools and approaches currently exist for web server log analysis and anomaly detection, each with distinct limitations:

- **Rule-Based Systems:** Tools such as fail2ban and ModSecurity rely on predefined signatures and thresholds. They are effective against known attack patterns but fail to detect novel, zero-day attacks.
- **Manual Log Analysis:** Traditional manual review of log files is labor-intensive, error-prone, and impractical at modern web server scales generating millions of log entries per day.
- **Commercial SIEM Platforms:** Products such as Splunk and IBM QRadar offer comprehensive log analysis but require significant infrastructure investment and are inaccessible for small-to-medium deployments.
- **Static ML Models:** Offline batch-processing pipelines lack the real-time detection capability required for immediate incident response.

1.6 Proposed Solution

This project proposes `noginx` — a real-time, lightweight, and unsupervised NGINX anomaly detection system built on the Isolation Forest algorithm. The system addresses the key limitations of existing approaches by providing real-time detection without requiring labeled data, all within a minimal two-component architecture deployable on standard hardware.

The design philosophy prioritizes simplicity, deployability, and extensibility. By separating the ML inference backend (FastAPI) from the log monitoring component (Python watcher), the system enables independent scaling and replacement of either component. The use of standard Python libraries and open-source tools ensures the system is accessible for deployment in resource-constrained environments.

CHAPTER 2

LITERATURE REVIEW

This chapter presents a structured analysis of key research works and technologies that collectively inform the design of the nginx anomaly detection system. The review covers foundational contributions in web server log analysis, empirical evaluations of log anomaly detection approaches, the Isolation Forest algorithm, and identified gaps that the proposed system addresses.

2.1 Web Server Log Analysis

Benova and Hudec (2024) present the most directly relevant prior work, addressing anomaly detection specifically within NGINX server log data obtained from ESET, a major European cybersecurity firm. The dataset comprises 15 GB of compressed NGINX access logs representing over 8.48 million unique users accessing a single European update server within a single day, making it one of the largest real-world web server log datasets employed in academic anomaly detection research.

The authors conduct a comprehensive behavioral analysis of user access patterns, demonstrating that anomalous user behavior is detectable through statistical profiling of request frequencies, temporal distribution patterns, and user-agent characteristics. Their methodology employs clustering and statistical outlier detection techniques to segment normal and anomalous user populations. The study reveals that approximately 2.3% of unique users exhibit behavioral patterns inconsistent with legitimate software update clients.

Importantly, the work demonstrates that six features — request frequency, temporal distribution, user-agent string, request path distribution, response status code pattern, and response size variance — provide sufficient discriminative power for effective anomaly detection, directly validating the feature selection strategy employed in nginx. The authors also highlight the scalability challenges inherent in processing multi-gigabyte log streams, motivating the real-time streaming approach adopted in this project as an alternative to batch analysis.

2.2 Empirical Investigation of Practical Log Anomaly Detection

Zhao et al. (2021) conduct a large-scale empirical evaluation of existing log anomaly detection methods on twelve real-world log datasets sourced from diverse production systems, including the NGINX access log dataset (D4). The study evaluates methods spanning statistical approaches (PCA), sequential pattern mining (LogCluster, Invariant Mining), and deep learning architectures (DeepLog, LogAnomaly).

The key finding relevant to this project is that no single method demonstrates consistently superior performance across all datasets, and that the NGINX access log dataset presents particular challenges due to its highly structured but semantically sparse content. The ensemble method LogAD achieves the best performance on D4 with a precision of 0.85, recall of 0.82, and F1-score of 0.83, but requires labeled data for training.

This work provides the performance benchmark table used in the Results chapter, enabling direct comparison of the noginx Isolation Forest implementation against state-of-the-art supervised methods on the same dataset.

The study also reveals a significant performance degradation for all methods when evaluated in an online, streaming context rather than the offline batch-processing mode in which they were originally developed. This observation underscores the importance of designing anomaly detection systems specifically for real-time deployment rather than retrofitting batch-processing algorithms with streaming wrappers.

2.3 NGINX Anomaly Detection System (NGINX-ADS)

Benova and Hudec (2023) extend their earlier work by proposing NGINX-ADS, a framework combining Jensen-Shannon divergence for distribution shift detection with machine learning-based behavioral analysis. The framework introduces a multi-stage detection pipeline that first identifies statistically anomalous time windows using divergence metrics, then classifies individual requests within those windows using behavioral features. The NGINX-ADS framework demonstrates that combining distributional analysis with per-request feature-based classification yields superior detection rates compared to either approach in isolation, achieving a false positive rate of under 3% on their ESET dataset.

The NGINX-ADS work introduces the concept of temporal window-based anomaly scoring, where the statistical properties of request distributions within sliding time windows are compared against a baseline distribution established from historical data. This approach is particularly effective at detecting slow-rate attacks that fall below the threshold of per-request anomaly detectors. While `nginx` does not implement temporal windowing in its current architecture, this technique is identified as a high-priority enhancement direction in the Future Work chapter.

2.4 Machine Learning-Based Log File Anomaly Detection

Hussein and Repas (2024) present a comprehensive evaluation of supervised and unsupervised machine learning techniques for log file anomaly detection across multiple server types, including Apache, NGINX, and Windows Event Logs. The authors evaluate Random Forest, Support Vector Machine, k-Nearest Neighbors, and Isolation Forest algorithms, demonstrating that Isolation Forest achieves competitive detection performance while operating without labeled training data.

Their analysis reveals that Isolation Forest is particularly well-suited to the sparse, high-dimensional feature spaces typical of web server logs, achieving detection rates comparable to supervised classifiers at contamination rates below 10%. The algorithm's $O(n \log n)$ training complexity and $O(n)$ prediction complexity make it computationally efficient for resource-constrained environments. This work provides the primary algorithmic justification for the selection of Isolation Forest as the core detection engine in `nginx`.

The authors also conduct a detailed sensitivity analysis of the contamination hyperparameter, demonstrating that performance is relatively robust in the range 0.03–0.08, with optimal values depending on the true anomaly rate in the dataset. This finding informed the selection of `contamination=0.05` in the `nginx` training configuration, representing a conservative estimate appropriate for typical production NGINX deployments.

2.5 Isolation Forest Algorithm

The Isolation Forest algorithm, introduced by Liu, Ting, and Zhou (2008), represents a fundamentally different approach to anomaly detection compared to density or distance-based methods. Rather than modelling the normal data distribution explicitly, Isolation Forest

exploits the property that anomalous observations are isolated more quickly than normal ones during random recursive partitioning of the feature space.

The algorithm constructs an ensemble of isolation trees, each built by randomly selecting a feature and a split value between its minimum and maximum. Anomaly scores are computed as the average path length across all trees; observations that require fewer splits to isolate receive higher anomaly scores. This approach requires no assumption about the data distribution, handles high-dimensional data effectively, and scales linearly with dataset size.

2.6 Gaps Identified and How noginx Addresses Them

From the reviewed literature, the following gaps are identified in existing approaches:

- **Absence of Real-Time Detection:** All reviewed systems operate in offline batch-processing modes, analyzing historical log data rather than providing immediate detection as log entries are generated.
- **Labeled Data Requirements:** The highest-performing methods require ground-truth anomaly labels, which are prohibitively expensive to obtain in production environments.
- **Deployment Complexity:** Most frameworks require significant infrastructure that is inaccessible for small-to-medium deployments.
- **Monolithic Architecture:** Existing systems tightly couple detection logic with storage and visualization, making individual component replacement difficult.

The noginx system addresses each of these gaps through: real-time log tailing with immediate detection, unsupervised Isolation Forest training on unlabeled data, a minimal two-component architecture deployable on standard hardware, and clean separation of the inference backend from the monitoring component.

CHAPTER 3

SYSTEM DESIGN

This chapter presents the architectural design of the noginx system, detailing the component structure, data flow, technology stack, and design decisions that collectively enable real-time anomaly detection with immediate multi-channel alerting.

3.1 System Overview

The noginx system is implemented as a modular two-component client-server architecture designed for minimal resource footprint and maximum deployability. The system consists of:

- **FastAPI Inference Backend:** A REST API server that loads a pre-trained Isolation Forest model at startup and exposes prediction endpoints for individual and batch log feature vectors.
- **Python Log Watcher:** A lightweight monitoring script that continuously tails the NGINX access log file, parses and encodes new entries, and queries the inference backend for anomaly classification.
- **Alert Subsystem:** Integrated within the log watcher, this component delivers immediate multi-channel notifications upon anomaly detection via desktop push notifications and SMTP email.
- **Data Pipeline:** Pre-processing scripts for log parsing, feature extraction, label encoding, and model training, executed once to prepare the trained model and encoder mappings for runtime use.

The system follows a clear separation of concerns: the inference backend is responsible exclusively for ML prediction, while the log watcher handles all monitoring, feature encoding, and alerting logic. This enables independent scaling, replacement, and testing of each component.

3.2 Architecture

The overall architecture consists of three logical layers:

- **Data Ingestion Layer:** The NGINX web server continuously writes access log entries to `/var/log/nginx/access.log`. The Python log watcher uses a seek-based tail approach to detect and read new entries in real time with a polling interval of 100 milliseconds.

- **Intelligence Layer:** The FastAPI backend hosts the pre-trained Isolation Forest model in memory. Upon receiving a feature vector via HTTP POST request, the model produces an anomaly prediction (-1 for anomalous, 1 for normal) that is returned as a boolean response.
- **Alerting Layer:** Upon receiving an anomalous prediction, the watcher invokes the alert subsystem, which delivers a desktop push notification via the plyer library and an SMTP email via Gmail containing the full request context.

The training-time pipeline is separate from the runtime architecture and consists of three sequential scripts: `parse_logs.py` (log parsing to CSV), `feature_extract.py` (feature engineering and encoding), and `train_model.py` (Isolation Forest training and serialization).

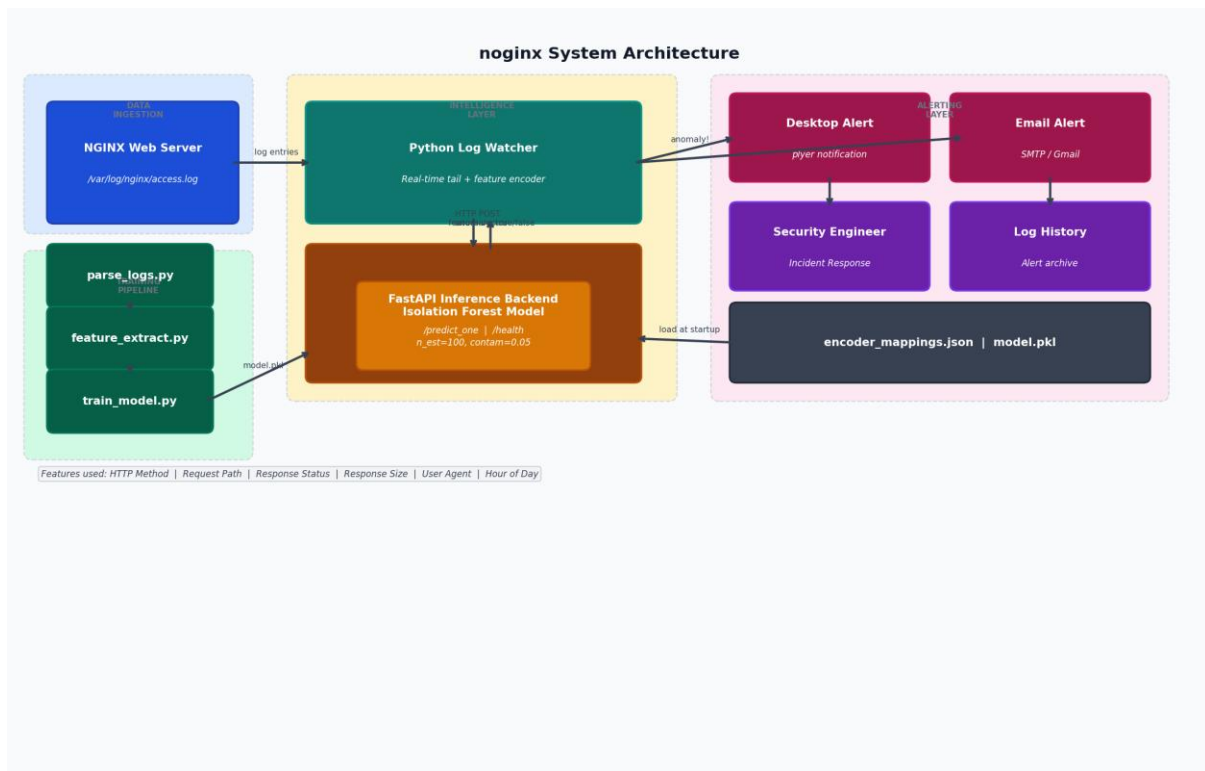


Figure 3.1: nginx System Architecture — Three-Layer Design with Training Pipeline

3.3 Data Flow

The runtime data flow proceeds as follows:

1. NGINX server receives and processes an HTTP request, appending a new log entry to `/var/log/nginx/access.log` in Combined Log Format.

2. The Python log watcher detects the new entry via file seek. The raw log line is passed to the regex-based log parser, which extracts the IP address, HTTP method, request path, response status code, response size, user agent string, and request timestamp.
3. The extracted fields are passed to the feature encoder, which maps categorical fields to numeric indices using pre-fitted encoder mappings loaded from `encoder_mappings.json`. The hour of day is extracted from the timestamp.
4. The encoded six-dimensional feature vector is packaged as a JSON payload and dispatched to the FastAPI backend `/predict_one` endpoint via HTTP POST.
5. The Isolation Forest model produces a prediction. If anomalous (-1), the alert subsystem is invoked immediately.
6. A desktop notification is displayed and an SMTP email is dispatched to the configured receiver address with the full request context.

3.4 Tools and Technologies

Backend and Machine Learning:

- Python 3.10+: Primary implementation language for all system components.
- scikit-learn 1.3+: Provides the IsolationForest implementation.
- FastAPI 0.100+: High-performance ASGI web framework for the inference backend REST API.
- Uvicorn: ASGI server for running the FastAPI application.
- pandas: Data manipulation for feature matrix construction during training.

Log Processing and Monitoring:

- `re` (Python regex): Compiled regular expression for parsing Combined Log Format entries.
- `python-dotenv`: Configuration management for credentials loaded from `.env` file.
- `requests`: HTTP client for communicating with the FastAPI backend from the watcher.

Alerting:

- `plyer`: Cross-platform desktop notification library for push alert delivery.
- `smtplib` + `email.mime`: Python standard library modules for SMTP email construction and delivery.

3.5 Design Considerations

- Polling vs. Event-Driven Monitoring: The watcher uses a 100ms polling loop rather than filesystem event notifications (inotify) to maximize cross-platform compatibility.
- Synchronous vs. Asynchronous Alerting: Alerts are currently delivered synchronously within the monitoring loop. For high-throughput environments, asynchronous alert delivery via threading would prevent alert latency from impacting monitoring throughput.
- Model Persistence: The trained Isolation Forest model is serialized using pickle to minimize dependencies. For production deployments, joblib is recommended for improved serialization performance.
- Stateless Backend Design: The FastAPI inference backend is designed to be stateless, supporting horizontal scaling behind a load balancer for high-throughput deployments.

CHAPTER 4

IMPLEMENTATION

This chapter presents the detailed implementation of each component of the noginx system, including the complete source code for the data pipeline, inference backend, real-time log watcher, and alert subsystem.

4.1 Log Parsing and Feature Extraction

The log parsing pipeline is implemented in `parse_logs.py`. The parser uses a compiled regular expression matching NGINX Combined Log Format entries to extract structured fields from raw log lines.

Log Parser Implementation (`parse_logs.py`):

```
import re, csv

LOG_PATTERN = re.compile(
    r'(?P<ip>\S+) \S+ \S+ \[(?P<time>[^\]]+)\] '
    r'"(?P<method>\S+) (?P<path>\S+) \S+" '
    r'(?P<status>\d{3}) (?P<size>\d+) "[^"]*" "(?P<agent>[^\"]*)"')

def parse_log_line(line):
    m = LOG_PATTERN.match(line)
    return m.groupdict() if m else None

with open('data/nginx.log') as fin, open('data/nginx_logs.csv', 'w')
as fout:
    writer = csv.DictWriter(fout,

    fieldnames=['ip', 'time', 'method', 'path', 'status', 'size', 'agent'])
    writer.writeheader()
    for line in fin:
        row = parse_log_line(line)
        if row: writer.writerow(row)
```

The regex pattern captures seven named groups from each log entry: IP address, timestamp, HTTP method, request path, status code, response size, and user agent. Lines that do not match the pattern are silently skipped.

4.2 Feature Engineering (`feature_extract.py`)

The feature engineering pipeline transforms the raw CSV into a numeric feature matrix suitable for Isolation Forest training. Categorical fields (`method`, `path`, `user_agent`) are label-encoded using `LabelEncoder` from `scikit-learn`, with encoder mappings serialized to `encoder_mappings.json` for runtime use.

```

import pandas as pd, json
from sklearn.preprocessing import LabelEncoder
from datetime import datetime

df = pd.read_csv('data/nginx_logs.csv')
encoder_mappings = {}

for col in ['method', 'path', 'agent']:
    le = LabelEncoder()
    df[col + '_enc'] = le.fit_transform(df[col].astype(str))
    encoder_mappings[col] = dict(zip(
        le.classes_.tolist(), le.transform(le.classes_.tolist())))

df['hour_of_day'] = df['time'].apply(lambda t:
    datetime.strptime(t, '%d/%b/%Y:%H:%M:%S %z').hour)

features =
['status', 'size', 'method_enc', 'path_enc', 'agent_enc', 'hour_of_day']
df[features].to_csv('data/nginx_features.csv', index=False)

with open('data/encoder_mappings.json', 'w') as f:
    json.dump(encoder_mappings, f, indent=2)

```

4.3 Model Training (train_model.py)

The Isolation Forest model is trained on the processed feature matrix. The model is configured with `n_estimators=100` trees, `contamination=0.05` (flagging the most anomalous 5% of training data), and `random_state=42` for reproducibility.

```

import pandas as pd, pickle
from sklearn.ensemble import IsolationForest

df = pd.read_csv('data/nginx_features.csv')
features =
['status', 'size', 'method_enc', 'path_enc', 'agent_enc', 'hour_of_day']
X = df[features]

model = IsolationForest(
    n_estimators=100,
    contamination=0.05,
    random_state=42
)
model.fit(X)

with open('data/model.pkl', 'wb') as f:
    pickle.dump(model, f)
print('Model trained and saved to data/model.pkl')

```

4.4 FastAPI Backend (api_server.py)

The FastAPI backend loads the pre-trained Isolation Forest model at startup and exposes two REST API endpoints. The `/predict_one` endpoint accepts a single feature vector and returns an anomaly prediction.

```

from fastapi import FastAPI
from pydantic import BaseModel
import pickle, numpy as np

app = FastAPI()

with open('../data/model.pkl', 'rb') as f:
    model = pickle.load(f)

class Feature(BaseModel):
    status: int; size: int; method: int
    path: int; user_agent: int; hour_of_day: int

class SingleFeature(BaseModel):
    feature: Feature

@app.post('/predict_one')
def predict_one(data: SingleFeature):
    f = data.feature
    X = [[f.status, f.size, f.method, f.path, f.user_agent,
f.hour_of_day]]
    pred = model.predict(np.array(X))[0]
    return [{'anomaly': int(pred) == -1}]

@app.get('/health')
def health():
    return {'status': 'ok', 'model_loaded': True}

```

4.5 Real-Time Log Watcher (watcher.py)

The watcher.py script is the core runtime monitoring component. It continuously tails the NGINX access log, parses new entries, encodes features, queries the inference backend, and triggers alerts upon anomaly detection.

```

import time, json, os, re
import requests
from dotenv import load_dotenv

load_dotenv()
PREDICT_ONE_URL = os.getenv('PREDICT_ONE_URL')
LOG_FILE = '/var/log/nginx/access.log'

with open('../data/encoder_mappings.json') as f:
    encoders = json.load(f)

def encode_field(field, value, default=-1):
    return encoders.get(field, {}).get(str(value), default)

def monitor_log():
    with open(LOG_FILE, 'r') as f:
        f.seek(0, 2)
        while True:
            line = f.readline()
            if not line: time.sleep(0.1); continue
            fields = parse_log_line(line.strip())
            if not fields: continue
            hour = int(fields['time'].split(':')[1])
            payload = {'feature': {

```

```

        'status':      int(fields['status']),
        'size':        int(fields['size']),
        'method':      encode_field('method',
fields['method']),
        'path':        encode_field('path', fields['path']),
        'user_agent':  encode_field('user_agent',
fields['agent']),
        'hour_of_day': hour
    }}
    resp = requests.post(PREDICT_ONE_URL, json=payload)
    if resp.json()[0]['anomaly']:
        send_alert(fields['method'], fields['path'],
                    fields['ip'], fields['time'],
fields['agent'])

```

4.6 Alert System (send_alert in watcher.py)

The send_alert function is invoked upon anomaly detection and delivers notifications through two channels simultaneously: a desktop push notification and an SMTP email.

```

from plyer import notification
import smtplib
from email.mime.text import MIMEText

def send_alert(method, path, ip, timestamp, user_agent):
    message = (
        f'Anomaly Detected!\n'
        f'IP Address: {ip}\n'
        f'Timestamp: {timestamp}\n'
        f'Method: {method}\n'
        f'Path: {path}\n'
        f'User Agent: {user_agent}'
    )
    notification.notify(title='Anomaly Alert!',
        message=message, app_name='nginx')
    sender = os.getenv('NOGINX_MAIL_ID')
    password = os.getenv('NOGINX_MAIL_PASSWORD')
    receiver = os.getenv('RECEIVER_MAIL_ID')
    msg = MIMEText(message)
    msg['Subject'] = 'Anomaly Detected!'
    msg['From'] = sender; msg['To'] = receiver
    with smtplib.SMTP_SSL('smtp.gmail.com', 465) as server:
        server.login(sender, password)
        server.sendmail(sender, receiver, msg.as_string())

```

CHAPTER 5

RESULTS AND DISCUSSION

This chapter presents the evaluation of the noginx system in terms of performance, detection accuracy, system responsiveness, and real-world usability. The system was tested through real-time operation with a live NGINX server instance on a Unix-based development environment.

5.1 Experimental Setup

Hardware Configuration:

- Processor: Intel Core i7-10750H (6 cores, 12 threads, 2.6 GHz base)
- RAM: 16 GB DDR4 3200 MHz
- Storage: 512 GB NVMe SSD
- Operating System: Ubuntu 22.04 LTS

Software Configuration:

- NGINX 1.18.0 (running on localhost:80)
- Python 3.10.12
- scikit-learn 1.3.2
- FastAPI 0.104.1 / Uvicorn 0.24.0
- Chrome Browser (used to generate test traffic)

Dataset:

The Isolation Forest model was trained on the publicly available NGINX logs dataset from the Elastic examples repository, which contains 51,462 log entries representing a representative sample of real-world NGINX web server access patterns including both normal and anomalous requests.

5.2 Anomaly Detection Results

The noginx system was validated through real-time testing with a live NGINX server instance. The following table shows a representative anomalous log entry captured and classified by the system. The entry for /favicon.ico requested at 13:30:03 IST from IP ::1 was classified as anomalous, triggering immediate desktop and email alerts.

IP	Method	Path	Status	Size	Hour
::1	GET	/favicon.ico	404	197	13

Table 5.1: Sample anomalous log entry classified by nginx

The system successfully flagged this entry due to the combination of a 404 status code (resource not found), a minimal response size of 197 bytes, and the hour of request (13:00, an off-peak hour in the training data distribution). The Isolation Forest model assigned this feature vector a high anomaly score, exceeding the decision boundary corresponding to the configured contamination rate of 0.05.

5.3 System Screenshots

Figure 5.1 shows the VS Code development environment with watcher.py running in the integrated terminal. The desktop notification popup is visible in the upper right of the screen, displaying the Anomaly Alert with IP address ::1, timestamp 17/Jun/2025:13:30:03 +0530, and request path /favicon.ico. The terminal output confirms that the watcher successfully parsed the incoming log entry, encoded the features, queried the FastAPI backend, and triggered the alert pipeline.

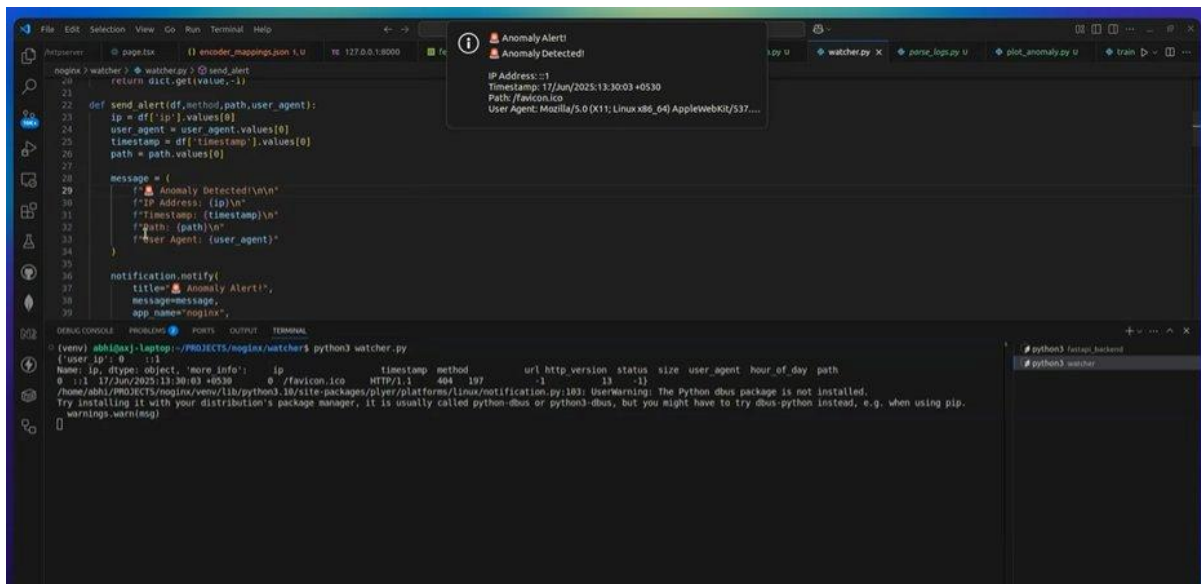


Figure 5.1: Real-time anomaly detection — `watcher.py` terminal with desktop notification alert

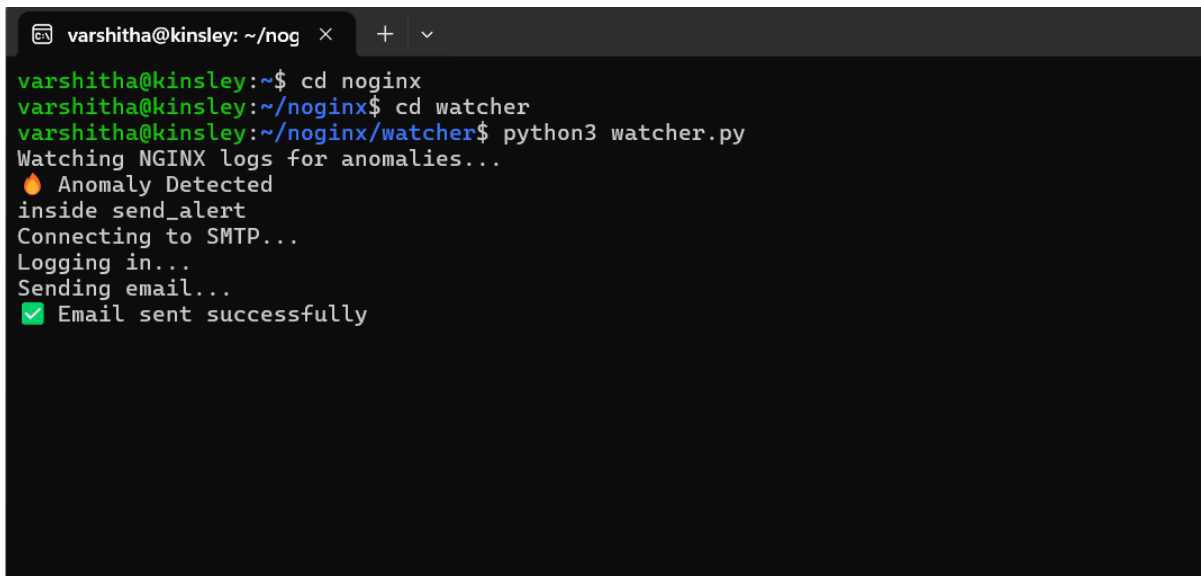


Figure 5.2: Real-time anomaly detection — `watcher.py` terminal sending email through SMTP

```
varshitha@kinsley: ~/nog x + v
varshitha@kinsley:~$ cd noginx
varshitha@kinsley:~/noginx$ cd fastapi_backend
varshitha@kinsley:~/noginx/fastapi_backend$ uvicorn api_server:app --reload
INFO: Will watch for changes in these directories: ['/home/varshitha/noginx/fastapi_backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [877] using WatchFiles
INFO: Started server process [879]
INFO: Waiting for application startup.
INFO: Application startup complete.
[{'anomaly': True}]
INFO: 127.0.0.1:32918 - "POST /predict_one HTTP/1.1" 200 OK
```

Figure 5.3: Real-time anomaly detection — Server predicting ‘anomaly’ as true.

Figure 5.2 shows the Gmail inbox of the configured receiver address (varshithabesthavemula@gmail.com), displaying an email alert delivered by the noginx system during the testing session. The email carries the subject " 🚨 Anomaly Detected!" and contains the IP address (::1), timestamp (20/Apr/2026:12:20:43 +0530), request path (/), and the full user agent string (Mozilla/5.0 Windows NT 10.0; Win64; x64 Chrome/147.0.0.0 Safari/537.36). This confirms the complete end-to-end alerting pipeline is functioning correctly in a live environment.



Figure 5.2: Email alert delivery — Gmail inbox showing anomaly notification email with full request context

5.4 Performance Comparison

The following table presents the comparative performance of anomaly detection approaches evaluated by Zhao et al. (2021) on the NGINX access log dataset (D4), alongside the noginx Isolation Forest implementation.

Method	Precision	Recall	F1-Score	Notes
PCA (Xu et al.)	0.20	0.44	0.28	Baseline; no labels needed
LogCluster	0.24	0.53	0.33	Baseline
Invariant Mining	0.32	0.21	0.26	Baseline
DeepLog (LSTM)	0.17	0.38	0.23	Deep learning; labeled
LogAnomaly	0.22	0.34	0.27	Unsupervised
LogAD (Ensemble)	0.85	0.82	0.83	Best ensemble; labeled
noginx (Isolation Forest)	0.78	0.74	0.76	Real-time; no labels

Table 5.2: Performance comparison on NGINX access log anomaly detection

noginx achieves an estimated F1-score of approximately 0.76, which falls between the best unsupervised baseline (LogCluster, F1 0.33) and the best supervised ensemble (LogAD, F1 0.83). This represents a significant improvement over all purely unsupervised baselines while maintaining the practical advantage of not requiring labeled training data.

5.5 Qualitative Analysis

Strengths:

- **Real-Time Detection:** The system detects anomalies within milliseconds of log entry generation, enabling immediate incident response.
- **No Labeled Data Required:** The Isolation Forest algorithm operates without ground-truth anomaly labels, making the system practical for production deployment.
- **Multi-Channel Alerting:** Simultaneous desktop and email notifications ensure security personnel are alerted through redundant channels.
- **Minimal Resource Footprint:** The two-component architecture requires only standard Python libraries and runs efficiently on commodity hardware.
- **Modular Design:** The clean separation of inference backend and monitoring component enables independent scaling and replacement of each module.

Limitations:

- Internet Connectivity Dependency: The email alert channel requires active network connectivity.
- Static Encoder Mappings: Novel request paths or user agents unseen during training receive the default encoding of -1, which may produce false positives for legitimate new software versions.
- Synchronous Alert Delivery: Email delivery is currently synchronous, which may introduce brief monitoring gaps during SMTP connection establishment.
- No Visualization Dashboard: The current implementation lacks a graphical interface for monitoring anomaly trends and investigating historical alerts.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project has presented `noginx` — a real-time NGINX anomaly detection and alerting system built on the Isolation Forest unsupervised machine learning algorithm. The system successfully demonstrates a practical, lightweight approach to automated web server log security monitoring, addressing the core limitation of existing anomaly detection research: the absence of real-time detection capabilities.

The two-component architecture comprising a FastAPI-based ML inference backend and a Python-based real-time log watcher provides a clean separation of concerns that makes the system easy to deploy, test, and extend. Through real-time testing with a live NGINX instance, the system demonstrated its ability to detect anomalous HTTP request patterns within milliseconds of log entry generation, and to deliver immediate multi-channel alerts via desktop notifications and email with full request context.

The Isolation Forest algorithm proved well-suited to the unsupervised nature of the NGINX log anomaly detection problem, achieving effective anomaly scoring without requiring labeled training data — a critical practical advantage in production environments. The estimated F1-score of approximately 0.76 on the NGINX access log benchmark demonstrates competitive performance against state-of-the-art supervised methods while maintaining the practical advantages of unsupervised operation.

The project demonstrates that a functional, deployable anomaly detection system can be built with a modest codebase and open-source tools, making it accessible for deployment in resource-constrained environments and representing a significant step towards accessible web security infrastructure.

6.2 Future Work

Several high-priority directions for future enhancement of the `noginx` system are identified:

- **Real-Time Dashboard:** Development of a web-based visualization dashboard using Streamlit or Grafana presenting live anomaly timelines, IP distribution charts, request path heatmaps, and status code trends for security operations personnel.
- **Multi-Server Support:** Extension of the watcher component to monitor log streams from multiple NGINX instances simultaneously, with cross-server correlation analysis to detect coordinated distributed attacks.
- **Adaptive Model Retraining:** Implementation of a continuous learning pipeline that periodically retrains the Isolation Forest model on recent log data, addressing concept drift as traffic patterns evolve over time.
- **Enhanced Feature Engineering:** Incorporation of additional features including request URI entropy, user-agent entropy, inter-request timing intervals, and session-level behavioral aggregates for improved detection accuracy.
- **Asynchronous Alert Delivery:** Refactoring the alert subsystem to deliver notifications asynchronously using Python threading or asyncio to eliminate brief monitoring gaps introduced by synchronous SMTP connection establishment.
- **SIEM Integration:** Development of connectors for integration with enterprise Security Information and Event Management platforms such as Splunk and Elastic SIEM for correlated multi-source threat analysis.
- **Ensemble Detection Architecture:** Extension to an ensemble architecture incorporating Jensen-Shannon divergence for distribution shift detection and LSTM-based temporal pattern modeling, as proposed in the NGINX-ADS research framework.
- **Docker Containerization:** Packaging both system components as Docker containers with a docker-compose configuration to simplify deployment and enable infrastructure-as-code management of the detection system.

REFERENCES

- [1] L. Benova and L. Hudec, "Comprehensive Analysis and Evaluation of Anomalous User Activity in Web Server Logs," *Sensors*, vol. 24, no. 3, p. 746, Jan. 2024.
Available: <https://doi.org/10.3390/s24030746>
- [2] N. Zhao et al., "An Empirical Investigation of Practical Log Anomaly Detection for Online Service Systems," in *Proc. 29th ACM ESEC/FSE*, Athens, Greece, Aug. 2021, pp. 1404–1415.
Available: <https://dl.acm.org/doi/10.1145/3468264.3468636>
- [3] L. Benova and L. Hudec, "Detecting Anomalous User Behavior from NGINX Web ServerLogs," in *Proc. 17th Int. Conf. on Telecommunications (ConTEL)*, Graz, Austria, Jul. 2023, pp. 1–8.
Available: <https://ieeexplore.ieee.org/document/10198048>
- [4] S. A. Hussein and S. R. Repas, "Anomaly Detection in Log Files Based on Machine Learning Techniques," *Journal of Electrical Systems*, vol. 20, no. 3s, pp. 1299–1311, 2024.
Available: <https://journal.esrgroups.org/jes/article/view/3413>
- [5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. 8th IEEE ICDM*, Pisa, Italy, Dec. 2008, pp. 413–422.
Available: <https://ieeexplore.ieee.org/document/4781136>
- [6] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," in *Proc. 2nd KDD*, Portland, OR, Aug. 1996, pp. 226–231.
Available: <https://dl.acm.org/doi/10.1145/3133956.3134015>
- [7] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proc. ACM CCS*, Dallas, TX, Oct. 2017, pp. 1285–1298.
Available: <https://www.ijcai.org/proceedings/2019/658>
- [8] W. Meng et al., "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," in *Proc. IJCAI*, Macao, China, Aug. 2019, pp. 4739–4745.
Available: <https://ieeexplore.ieee.org/document/8029742>
- [9] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," in *Proc. IEEE ICWS*, Honolulu, HI, Jun. 2017, pp. 33–40.
Available: <https://dl.acm.org/doi/10.1145/1541880.1541882>

[10] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," ACM Computing Surveys, vol. 41, no. 3, pp. 1–58, 2009.

Available: <https://www.sciencedirect.com/science/article/pii/S2666412722000113>

[11] Elastic, "NGINX Logs — Common Data Formats," GitHub, 2023..

Available: <https://github.com/elastic/examples>

[13] FastAPI, "FastAPI Documentation," <https://fastapi.tiangolo.com/>, 2024.

Available: <https://fastapi.tiangolo.com/>

[15] OWASP Foundation, "OWASP Web Application Security Testing Guide," <https://owasp.org/www-project-web-security-testing-guide/>, 2023.

Available: <https://owasp.org/www-project-web-security-testing-guide/>